

Host Discovery con Scapy

Realizado por:

- Nombre: Juan José Ibáñez
- Curso: 2º Ingeniería

Objetivo

El objetivo de esta práctica es implementar un mecanismo de **host discovery** utilizando la librería Scapy en Python. Se pretende detectar hosts activos mediante el envío de paquetes usando diferentes protocolos de red: UDP, TCP (ACK) e ICMP (timestamp request).

Además, se busca analizar las respuestas obtenidas para determinar si un host está activo en la red.

Código principal

```
from scapy.all import ICMP, IP, TCP, UDP

def craft_discovery_pkts(protocols, ip_range, pkt_counts=None, port=80):

    if isinstance(protocols, str):
        protocols = [protocols]

    pkt_counts = pkt_counts or {}

    packets = []

    for proto in protocols:

        proto_lower = proto.lower().strip()

        count = pkt_counts.get(proto_lower, 1)

        for i in range(count):
            if proto_lower == "udp":

                pkt = IP(dst=ip_range) / UDP(dport=port)

            elif proto_lower == "tcp":

                pkt = IP(dst=ip_range) / TCP(dport=port, flags="S")

            elif proto_lower == "icmp":
                pkt = IP(dst=ip_range) / ICMP(type=8)

            else:
                continue

            packets.append(pkt)

    return packets
```

Código Main

```
from scapy.all import IP, ICMP, TCP, UDP, sr1, conf, IPError
from host_discovery import craft_discovery_pkts
import logging

logging.getLogger("scapy.runtime").setLevel(logging.ERROR)
```

```

def host_discovery(target_ip, protocols, timeout=2):

    packets = craft_discovery_pkts(protocols, target_ip)
    results = {}

    for pkt in packets:

        if ICMP in pkt:
            proto = "icmp"
        elif TCP in pkt:
            proto = "tcp"
        else:
            proto = "udp"

        answered = sr1(pkt, timeout=timeout, verbose=0)

        responded = False

        if answered and IP in answered:
            src_ip = answered[IP].src

            if src_ip == target_ip:

                if proto == "icmp":
                    if answered.haslayer(ICMP):
                        if answered[ICMP].type in [0, 14]:
                            responded = True

                elif proto == "tcp":
                    if answered.haslayer(TCP):
                        flags = answered[TCP].flags
                        if flags & 0x02 or flags & 0x04:
                            responded = True

                elif proto == "udp":

                    if answered.haslayer(UDP):
                        responded = True

            elif answered.haslayer(ICMP) and answered[ICMP].type == 3:

                if answered.haslayer(IPError) and answered[IPError].dst == target_ip:

                    if src_ip == target_ip:
                        responded = True

        results[proto] = responded
        status = "SI" if responded else "NO"
        print(f"IP: {target_ip:<15} | Protocolo: {proto.upper():<4} | Responde: {status}")

    return results

def main():
    print("\n" + "="*30)
    print("  Host Discovery")
    print("="*30)

```

```

ips_input = input("Introduce IPs separadas por coma: ")
targets = [ip.strip() for ip in ips_input.split(",") if ip.strip()]

prot_input = input("Protocolos (icmp,tcp,udp) [Enter = todos]: ").strip().lower()

if prot_input == "":
    protocols = ["icmp", "tcp", "udp"]
else:
    protocols = [p.strip() for p in prot_input.split(",") if p.strip()]

active_hosts = []

for ip in targets:
    print(f"\n--- Analizando: {ip} ---")
    res = host_discovery(ip, protocols)

    if any(res.values()):
        active_hosts.append(ip)

print("\n" + "="*30)
print("      RESULTADO FINAL")
print("="*30)
if active_hosts:
    print(f"Hosts activos confirmados: {' '.join(active_hosts)}")
else:
    print("No se detectó actividad de ningún host.")

if __name__ == "__main__":
    main()

```

Ejemplo de uso

El script se ejecuta con privilegios de administrador debido al uso de Scapy:

```
sudo python main.py
```

En este ejemplo se analizan dos direcciones IP:

10.47.135.178 → host activo 10.47.135.170 → host inactivo

Se envían paquetes utilizando los protocolos ICMP, TCP y UDP, y se analiza si se obtiene respuesta.

Resultados

Durante la ejecución se obtuvieron los siguientes resultados:

La IP 192.168.1.1 respondió a uno o varios protocolos, por lo que se considera un host activo. La IP 192.168.1.250 no respondió a ningún protocolo, por lo que se considera inactiva.

Se observa que:

ICMP suele ser efectivo si el host no bloquea este tipo de tráfico. TCP ACK puede generar respuestas RST en hosts activos. UDP no siempre genera respuesta, pero puede provocar mensajes ICMP si el host existe.

Finalmente, el script identifica correctamente los hosts activos en función de las respuestas recibidas.

Captura de Wireshark

A continuación se muestra una captura de Wireshark donde se observan los paquetes enviados y las respuestas obtenidas:

Capturing from veth9e33891						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	02:42:86:19:c4:2e	Broadcast	ARP	42	Who has 172.17.0.2? Tell 172.17.0.1
2	0.000046145	02:42:ac:11:00:02	02:42:86:19:c4:2e	ARP	42	172.17.0.2 is at 02:42:ac:11:00:02
3	0.027574104	172.17.0.1	172.17.0.2	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (reply in 4)
4	0.027608108	172.17.0.2	172.17.0.1	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (request in 3)
5	0.067905548	172.17.0.1	172.17.0.2	TCP	54	20 → 80 [SYN] Seq=0 Win=8192 Len=0
6	0.068020704	172.17.0.2	172.17.0.1	TCP	58	80 → 20 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
7	0.068048336	02:42:86:19:c4:2e	Broadcast	ARP	42	Who has 172.17.0.2? Tell 172.17.0.1
8	0.068053316	02:42:ac:11:00:02	02:42:86:19:c4:2e	ARP	42	172.17.0.2 is at 02:42:ac:11:00:02
9	0.068067033	172.17.0.1	172.17.0.2	TCP	54	20 → 80 [ACK] Seq=1 Win=0 Len=0
10	0.112518643	172.17.0.1	172.17.0.2	UDP	42	53 → 80 Len=0
11	0.112551445	172.17.0.2	172.17.0.1	ICMP	70	Destination unreachable (Port unreachable)

- Frame 1: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface veth9e33891, id 0000 - Ethernet II, Src: 02:42:86:19:c4:2e (02:42:86:19:c4:2e), Dst: Broadcast (ff:ff:ff:ff:ff:ff) - Address Resolution Protocol (request)	<pre> 0000 ff ff ff ff ff 02 42 86 19 c4 2e 08 06 00 01 B 0010 08 00 06 04 00 01 02 42 86 19 c4 2e ac 11 00 01 B 0020 00 00 00 00 00 00 ac 11 00 02 </pre>
--	---

Se pueden identificar:

Paquetes ICMP timestamp request Paquetes TCP con flag ACK Paquetes UDP enviados al puerto 80

Respuestas del host activo